Demo: Materialized View

Summary

Created a Materialized View named `tickets_mv` in Amazon Redshift using sales and event data, validated cached query results, updated underlying table data, refreshed the Materialized View manually, and confirmed that refreshed results reflected the latest sales values successfully.

What is a Materialized View?

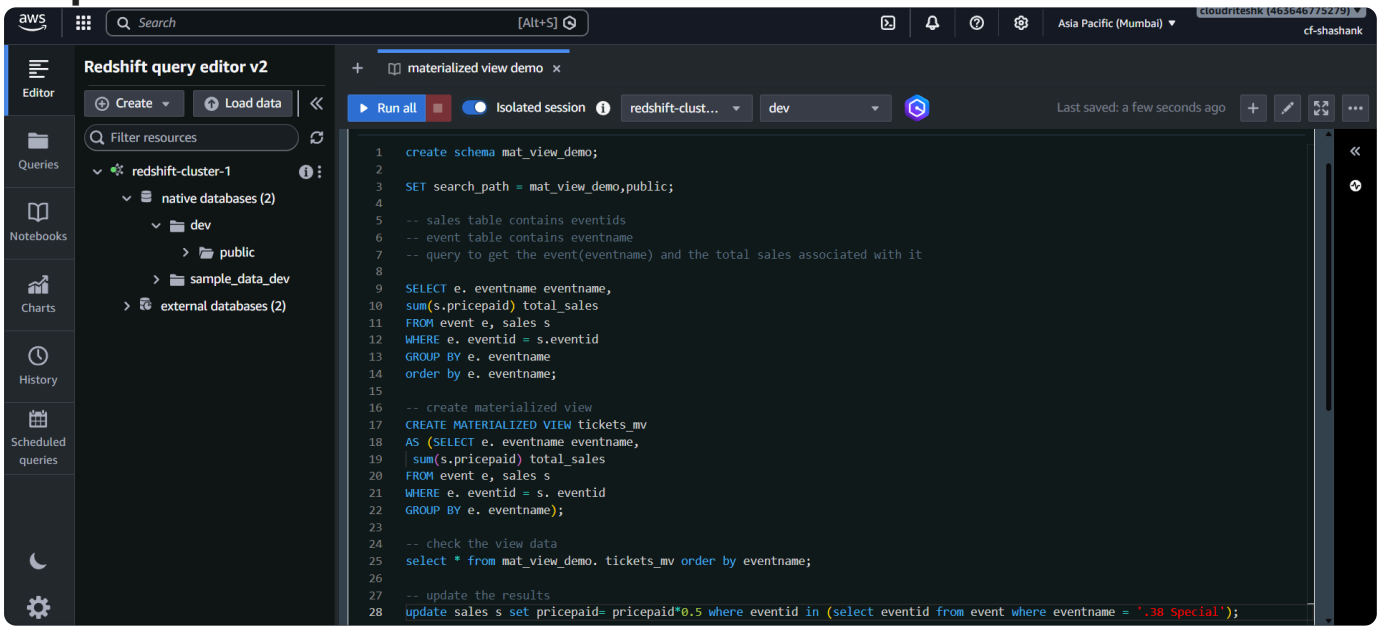
A Materialized View:

- Stores query results physically
- Helps improve performance for repetitive queries
- Avoids executing complex joins repeatedly

Useful for:

- Dashboards
- Reports
- Frequently accessed analytics queries

Step 1 – Create Schema



Created a separate schema for demo data:

```
CREATE SCHEMA mat_view_demo;
```

Purpose:

- Keep materialized view objects separate from other database objects.

Step 2 – Set Search Path

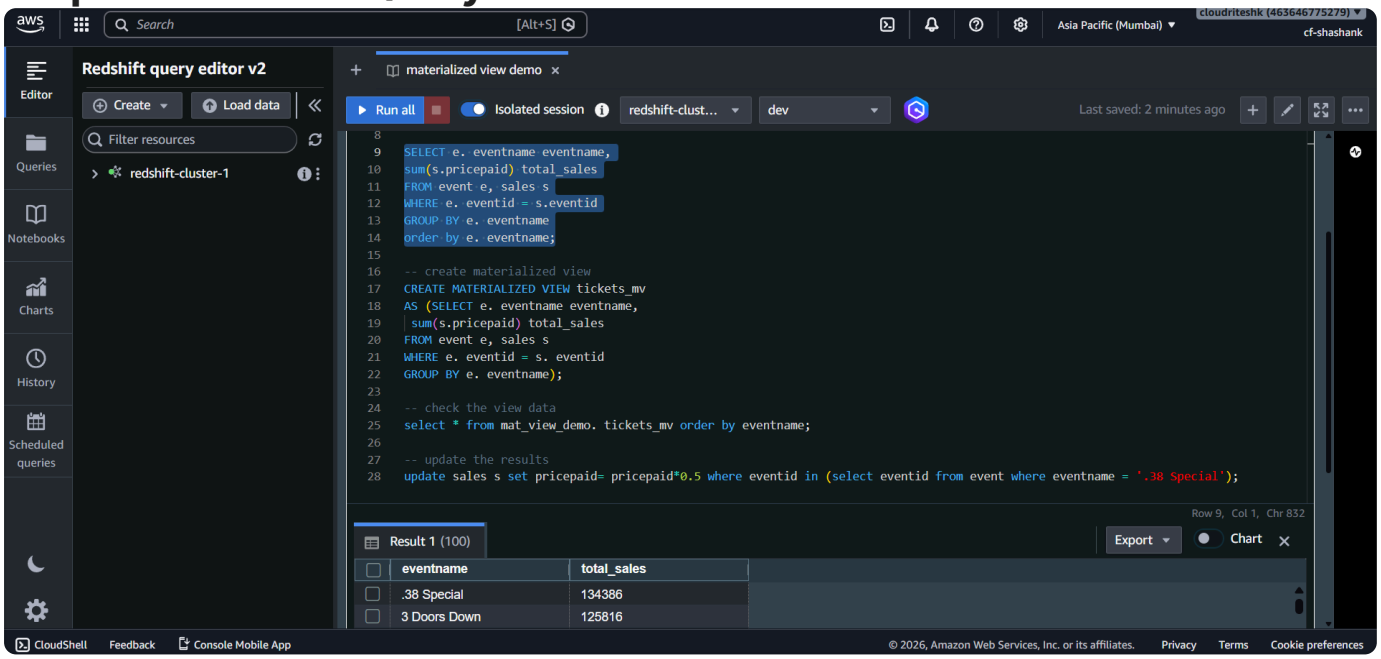
Used:

```
SET search_path
```

Purpose:

- Avoid writing schema prefixes repeatedly in queries.

Step 3 – Run Base Query



The query:

- Joined:
  - `sales`
  - `event`
- Returned:
  - Event name
  - Associated total sales

Why JOIN?

- `sales` table contains:
  - `eventid`
  - sales values
- `event` table contains:
  - event names

The query returned: `100 rows`

Why Use Materialized View?

This query may:

- Return large datasets
- Be executed multiple times daily
- Be used by dashboards and reports

Instead of running the query every time:

- Results can be cached using a Materialized View.

This improves:

- Query performance
- Dashboard response time

Step 4 – Create Materialized View

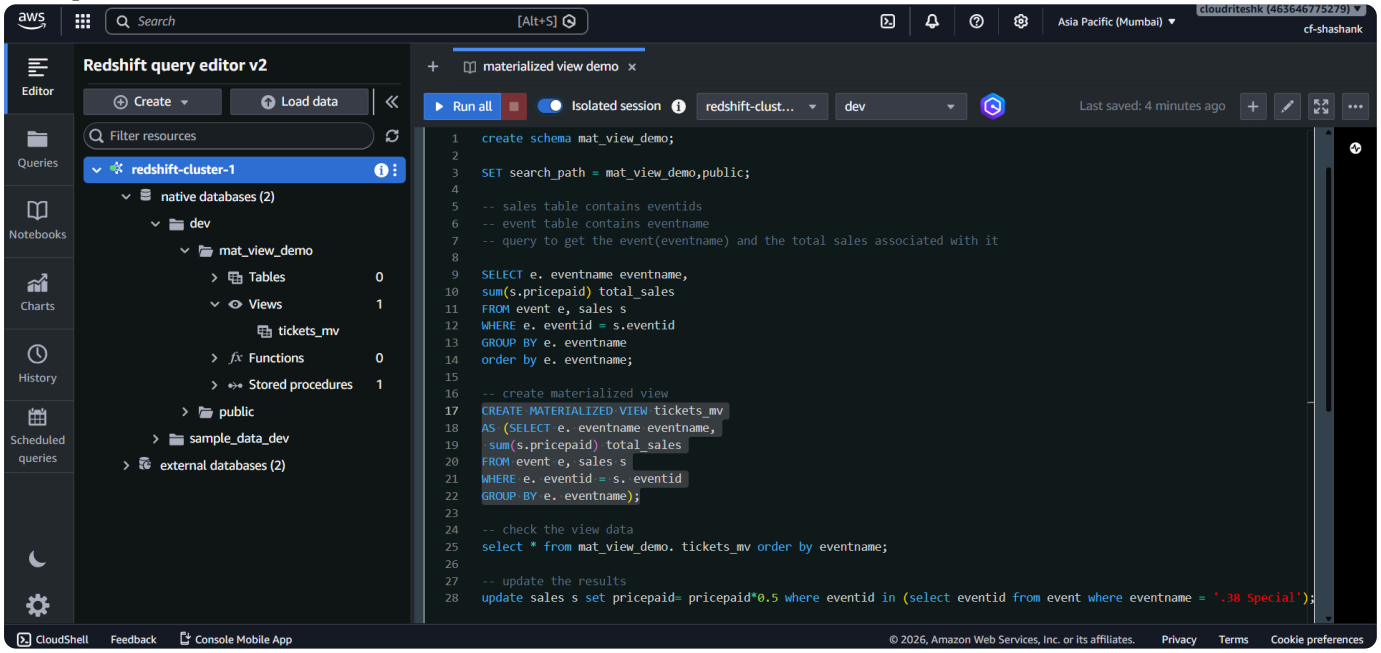
Used:

```
CREATE MATERIALIZED VIEW
```

Materialized view name: `tickets_mv`

The previously executed query was used inside the view definition.

Step 5 – Validate Materialized View



After refreshing the left panel:

Inside schema: `mat_view_demo`

Observed:

- 1 View
- 1 Internal Table

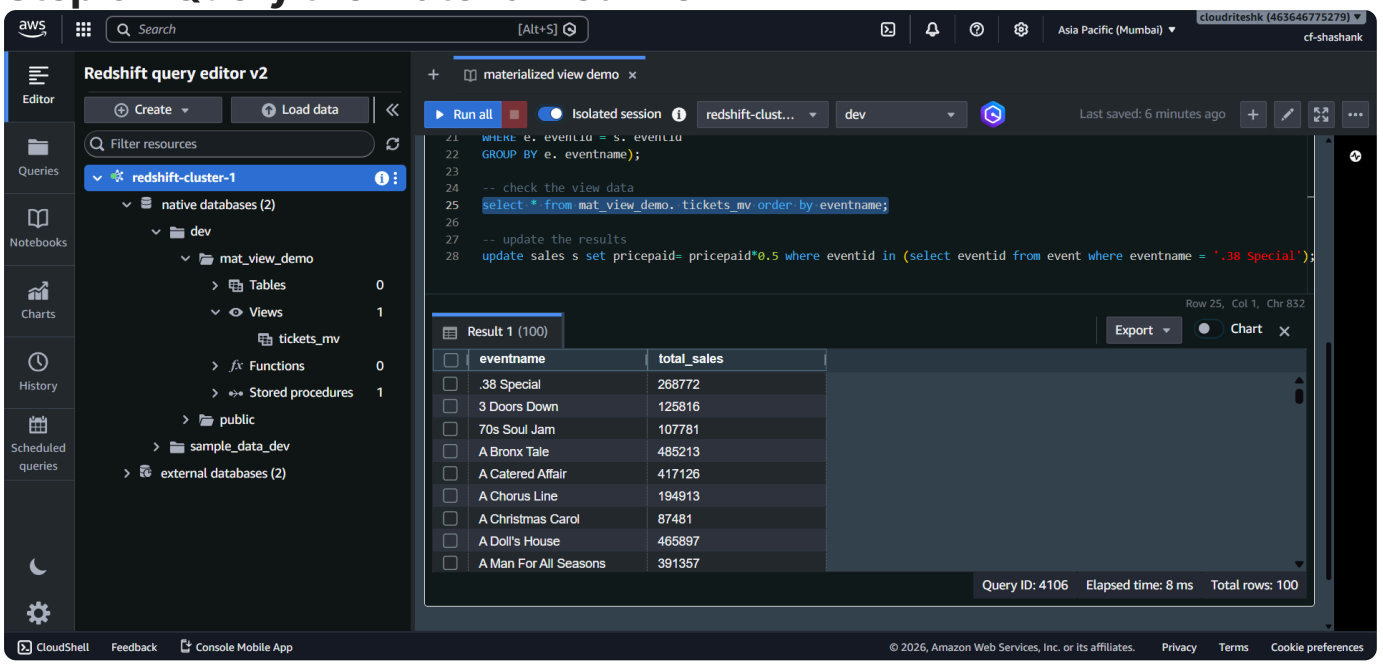
Important Note

The internal table:

- Is managed automatically by Redshift
- Stores cached data internally
- Is not directly used by users

We interact only with: `tickets_mv`

Step 6 – Query the Materialized View



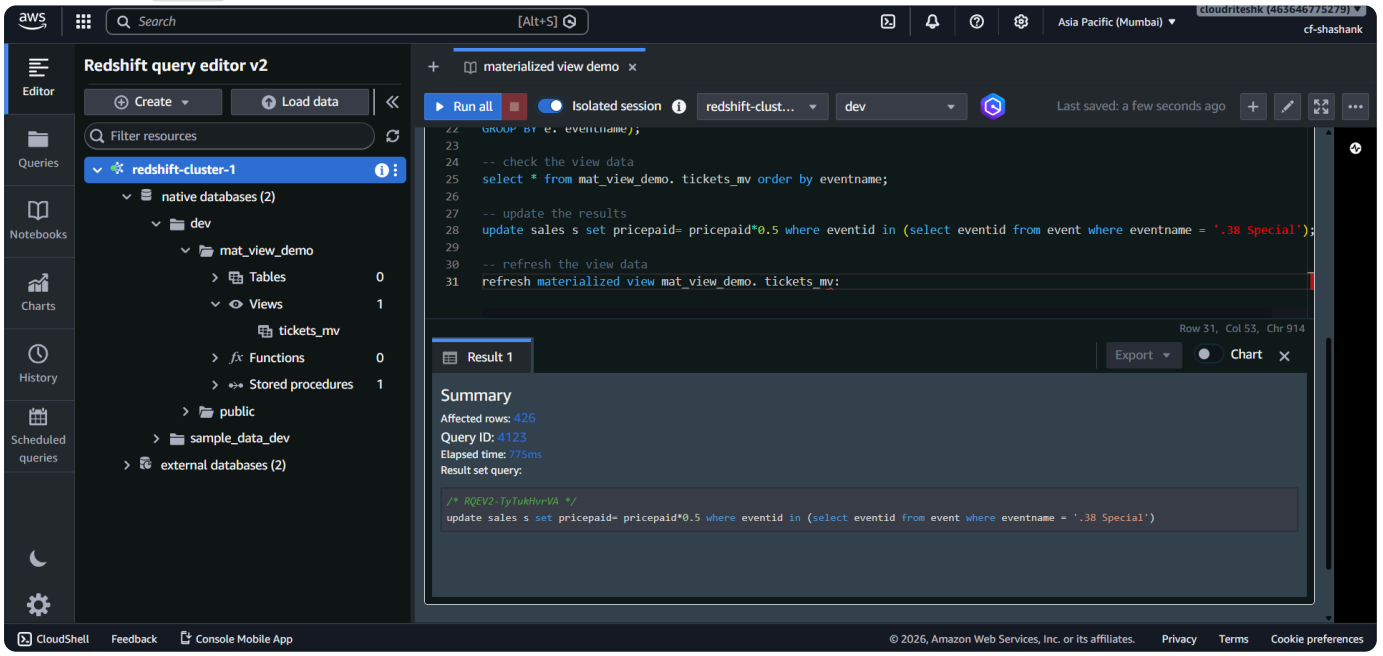
Executed query on: `tickets_mv`

Result: `100 rows`

Same output as the original query was returned successfully.

Step 7 – Update Base Table Data

Updated the `sales` table.



Example Used

Event: `0.38 Special`

Action:

- Doubled the sales amount for all related rows.

Expected sales:

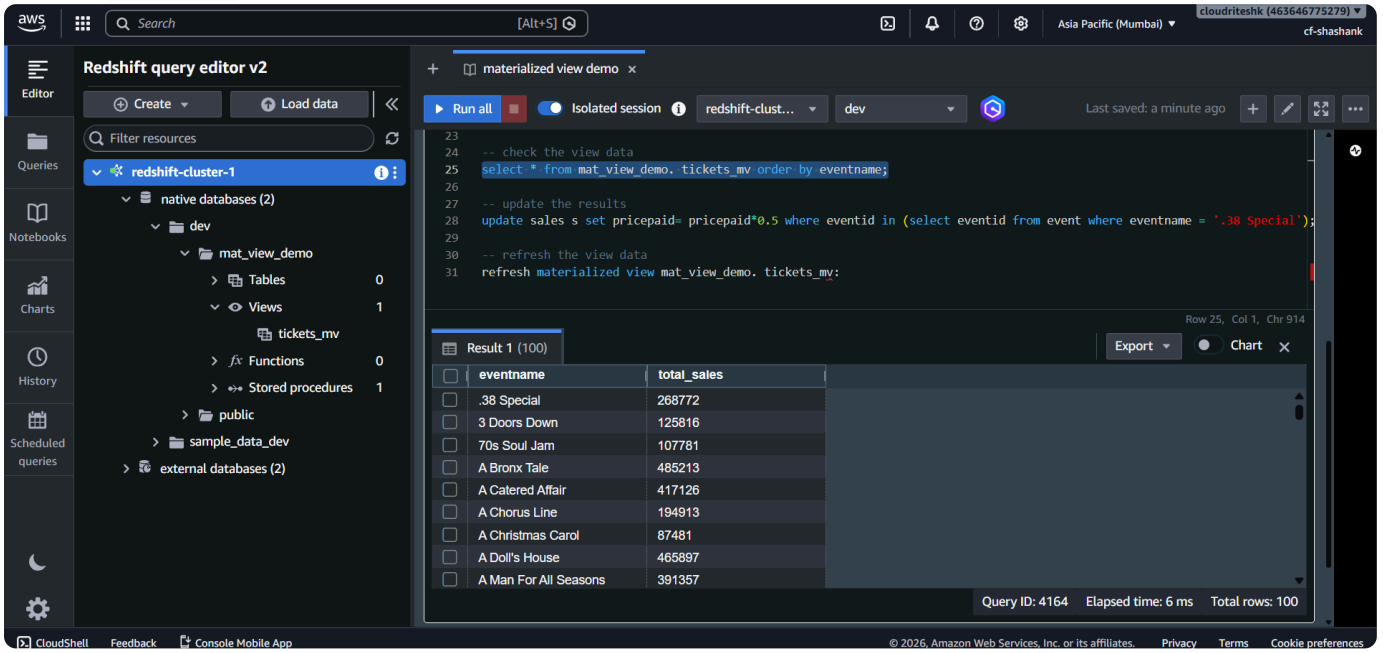
- Increased from: `~0.26 million`

to: `~0.5 million`

Updated rows: `426 rows`

Step 8 – Check Materialized View Data

Queried: `tickets_mv`



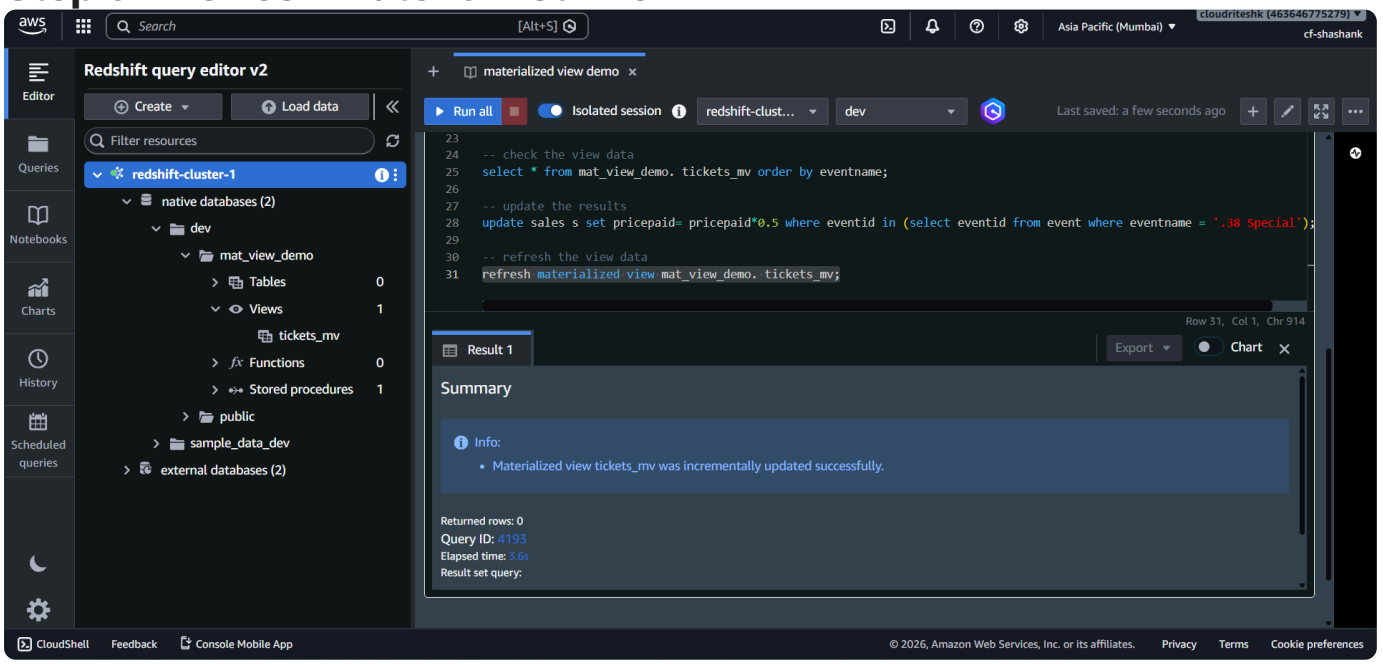
Observation:

- Old values were still returned.

Why?

Materialized Views use cached data.
They do not automatically refresh after table updates unless refreshed explicitly.

Step 9 – Refresh Materialized View



Used:

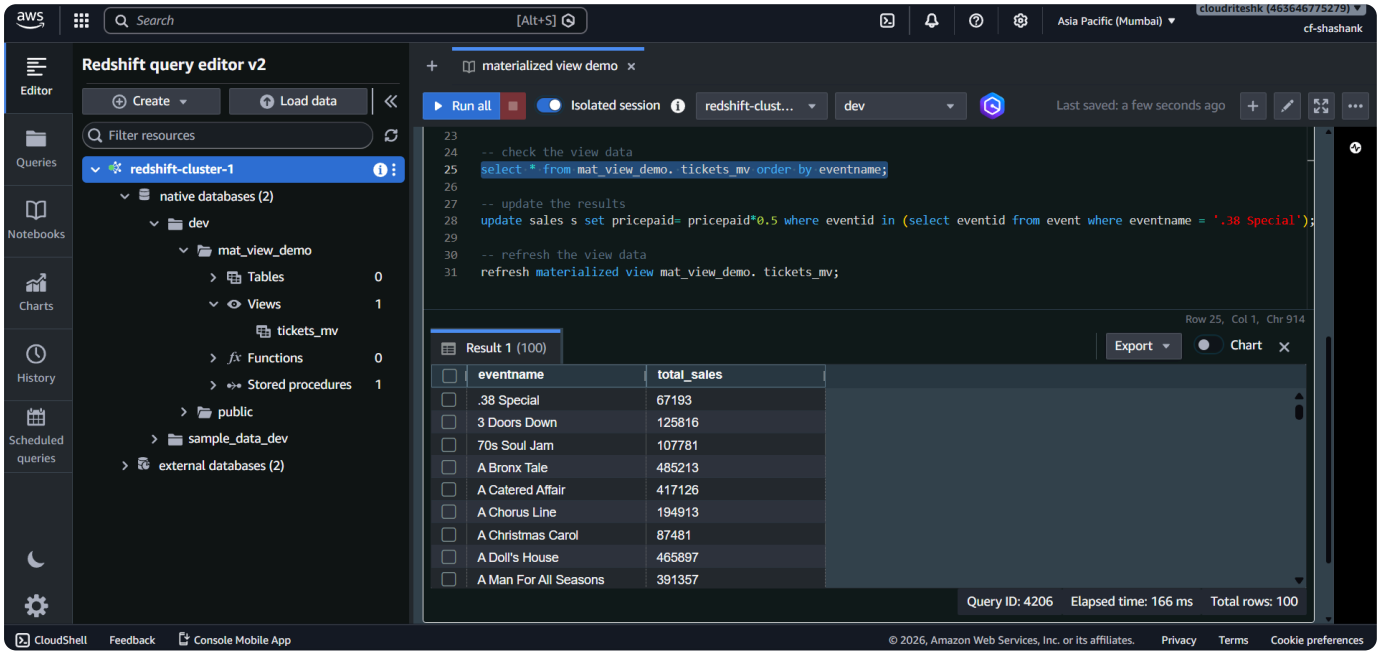
```
REFRESH MATERIALIZED VIEW
```

Purpose:

- Reload latest data from base tables.

Step 10 – Validate Refreshed Data

Queried the Materialized View again.



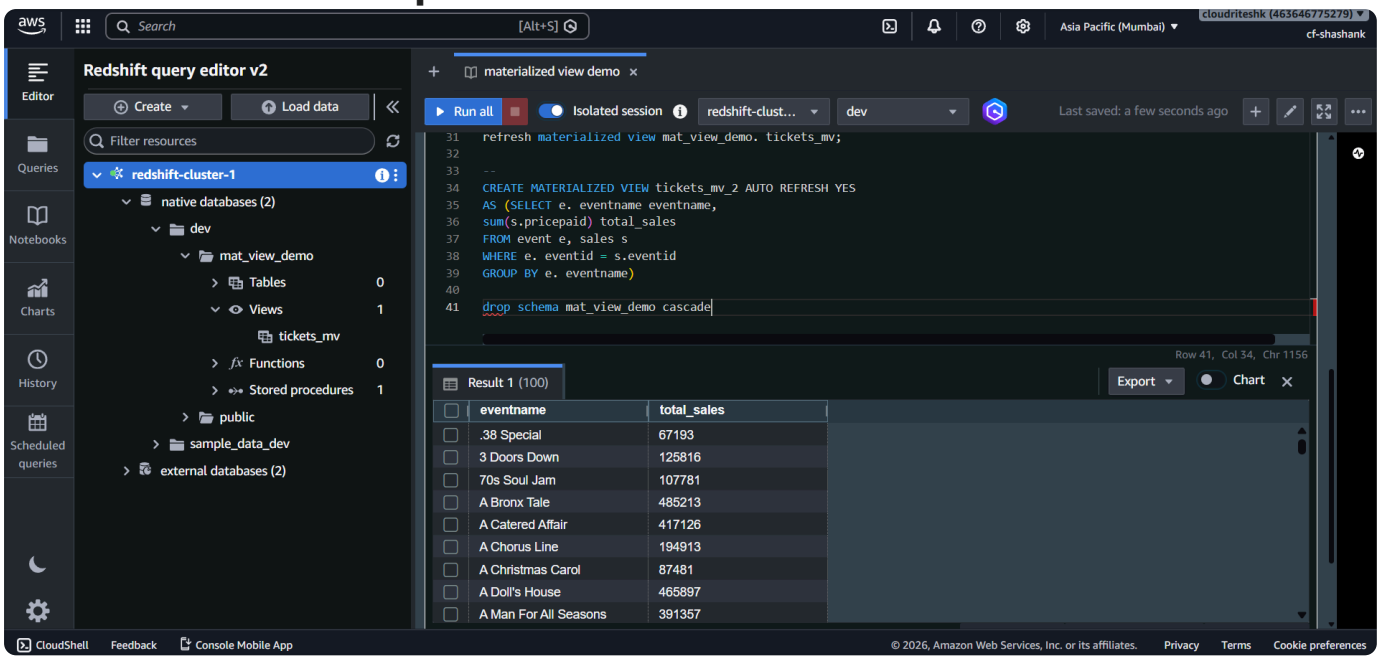
Result:

- Event: `0.38 Special`

now showed: `0.5+ million sales`

This confirmed the refresh worked successfully.

Automatic Refresh Option



Materialized Views can also be configured for:

- Automatic refresh

With this:

- Redshift refreshes the view automatically after detecting table updates.
- Refresh is optimized to avoid affecting analytic workloads.

Key Learning

Materialized Views help:

- Cache expensive query results
- Improve analytics performance
- Reduce repeated query execution time

Best suited for:

- Reporting queries
- BI dashboards
- Repetitive aggregations and joins
